Actividad 5 — Playwright

Testing automatizado, QA adaptativo e Inteligencia Artificial aplicada a aplicaciones SPA

Introducción

El testing automatizado ocupa un lugar importante dentro del desarrollo de software actual, especialmente en aplicaciones web que poseen interfaces dinámicas y una interacción constante con el usuario. En este contexto, herramientas como Playwright permiten automatizar acciones dentro del navegador y comprobar de manera sistemática que una aplicación responda de acuerdo con el comportamiento esperado.

La actividad desarrollada se centra en Playwright y en su aplicación al testing End-to-End de una Single Page Application. Además, se analiza el concepto de QA adaptativo, las dificultades que presentan las SPA para los mecanismos tradicionales de automatización, la incorporación de modelos de lenguaje como complemento para la generación y adaptación de pruebas, y finalmente la implementación y ejecución de una suite concreta de tests. La presentación se organiza en cinco módulos: introducción a Playwright; QA adaptativo y desafíos en SPA; Playwright e Inteligencia Artificial; implementación y ejecución de tests E2E; y especificación de testing y selección de la SPA.

---

1. Introducción a Playwright

Playwright es una herramienta de automatización utilizada para probar aplicaciones web mediante navegadores reales. Permite simular acciones que normalmente realizaría una persona, como ingresar a una página, completar campos, presionar botones, navegar entre distintas secciones de una aplicación y comprobar los resultados producidos por esas acciones.

Uno de sus principales usos es el testing End-to-End o E2E. Este enfoque busca validar un flujo funcional completo desde el punto de vista del usuario, en lugar de comprobar únicamente funciones o componentes individuales. De esta forma, una prueba puede comenzar con el acceso a una aplicación, continuar con distintas interacciones y finalizar verificando que el sistema haya llegado al estado esperado.

Entre las características principales trabajadas en la actividad se encuentran la automatización de acciones de usuario, la posibilidad de ejecutar pruebas sobre Chromium, Firefox y WebKit, el uso de Locators, mecanismos de auto-waiting y assertions, además de la posibilidad de utilizar los mismos tests en diferentes navegadores.

Un test sencillo puede, por ejemplo, ingresar a una dirección determinada mediante `page.goto()`, localizar un campo de usuario, completar información, encontrar el botón correspondiente al ingreso y hacer clic sobre él. Finalmente, mediante una assertion se comprueba que la aplicación haya alcanzado el resultado esperado. Por lo tanto, una prueba automatizada no se limita a reproducir acciones, sino que también debe verificar que el resultado producido sea correcto.

Comparación con Selenium y Cypress

Dentro del testing web existen distintas herramientas de automatización. La actividad compara Playwright con Selenium y Cypress teniendo en cuenta aspectos como su enfoque, navegadores, sincronización, lenguajes disponibles y mecanismos de depuración.

Playwright se presenta como una herramienta orientada al testing E2E y a la automatización de aplicaciones web modernas. Trabaja con Chromium, Firefox y WebKit, utiliza auto-waiting junto con Locators, admite JavaScript/TypeScript, Python, Java y .NET, y dispone de herramientas como Trace Viewer y HTML Report.

Selenium se orienta a la automatización web mediante WebDriver y posee compatibilidad con navegadores como Chrome, Edge, Firefox y Safari. El manejo de la sincronización se realiza mediante estrategias de espera y cuenta con soporte para diferentes lenguajes, entre ellos Java, Python, JavaScript, C#, Ruby y Kotlin.

Cypress también se encuentra orientado al testing de aplicaciones web modernas y al Component Testing. Su mecanismo de sincronización se apoya en retry-ability, trabaja principalmente con JavaScript y TypeScript y dispone de un runner interactivo junto con herramientas de registro y depuración. En la comparación realizada, el soporte para WebKit se presenta como experimental.

La elección entre estas herramientas depende de las características del proyecto, el entorno tecnológico utilizado, los navegadores que se necesiten cubrir y los requerimientos particulares del equipo de desarrollo y QA.

---

2. QA adaptativo y desafíos de las SPA

El concepto de QA adaptativo hace referencia a pruebas capaces de resistir o adaptarse de manera controlada a determinados cambios que se producen en la aplicación.

Un esquema de testing demasiado rígido puede fallar ante modificaciones relativamente pequeñas de la interfaz. Por ejemplo, si cambia la estructura del DOM, un selector utilizado por una prueba puede dejar de encontrar un determinado elemento aunque la funcionalidad continúe siendo válida desde el punto de vista del usuario.

El QA adaptativo busca reducir este problema mediante una combinación de automatización, selectores robustos, análisis del DOM y, eventualmente, técnicas de

Inteligencia Artificial. Frente a una modificación de la interfaz, el sistema puede analizar el nuevo estado, buscar una alternativa válida y posteriormente comprobar que esa adaptación sea correcta.

Single Page Applications

Una Single Page Application o SPA es una aplicación en la que gran parte de la navegación y actualización de la interfaz se realiza de manera dinámica mediante JavaScript, sin que sea necesario recargar completamente la página para cada interacción.

Tecnologías como Angular, React y Vue son ejemplos asociados al desarrollo de este tipo de aplicaciones. En una SPA, el usuario puede realizar una acción, JavaScript modificar el estado interno y posteriormente actualizar el DOM sin que se produzca una recarga completa de la página.

Esta característica introduce tres desafíos principales para el testing automatizado: los selectores frágiles, el contenido asincrónico y las rutas client-side.

Selectores frágiles

Un selector frágil es aquel que depende excesivamente de la estructura exacta del DOM.

Por ejemplo, un selector que intenta localizar un determinado botón según su posición dentro de varios elementos `<div>` puede dejar de funcionar si un desarrollador modifica la estructura de la página, incluso cuando el botón continúa existiendo y funcionando correctamente.

Frente a este problema, Playwright permite utilizar Locators más robustos, capaces de identificar un elemento a partir de su función y nombre. En la actividad se compara un selector dependiente de la estructura interna con un Locator basado en `getByRole`, donde el elemento se identifica como un botón llamado “Guardar”.

De esta manera, los tests pueden reducir su dependencia respecto de modificaciones puramente estructurales de la interfaz.

Contenido asincrónico

El contenido asincrónico representa otro desafío habitual.

Una aplicación puede recibir una acción del usuario, realizar una solicitud a una API, esperar la respuesta, modificar el DOM y posteriormente mostrar los resultados. El tiempo necesario para completar este proceso puede variar entre distintas ejecuciones.

Utilizar una espera fija como:

await page.waitForTimeout(2000);

constituye un enfoque poco robusto, ya que supone que dos segundos serán siempre suficientes.

En cambio, una estrategia más apropiada es esperar hasta que se cumpla una condición concreta, por ejemplo que aparezca un determinado texto:

```
await expect(
  page.getByText('Resultado encontrado')
).toBeVisible();
```

De esta forma, la prueba no depende de una cantidad arbitraria de tiempo, sino del estado real de la aplicación.

Rutas client-side

En una SPA también es posible navegar entre diferentes rutas sin realizar una recarga completa del documento.

Por ejemplo, la aplicación puede pasar de `/productos` a `/productos/15` y posteriormente a `/carrito`, mientras JavaScript se encarga de modificar la URL y actualizar los componentes visibles.

Esto significa que la URL y la interfaz pueden cambiar sin que exista una recarga completa de la página, por lo que las pruebas deben tener en cuenta el estado real de la aplicación y no depender únicamente de mecanismos tradicionales de carga.

La relación entre una SPA dinámica, el testing automatizado y el QA adaptativo permite avanzar hacia tests más resistentes y mantenibles.

---

3. Playwright e Inteligencia Artificial

El tercer módulo analiza la posibilidad de complementar Playwright con Inteligencia Artificial y, particularmente, con un modelo de lenguaje o LLM.

La idea planteada consiste en dividir responsabilidades: el modelo de lenguaje interpreta la situación, analiza el contexto y propone qué acción resulta conveniente, mientras que Playwright se encarga de ejecutar las acciones concretas sobre la aplicación y permite verificar posteriormente lo ocurrido.

Esta combinación puede representarse mediante un ciclo de trabajo en el que un agente de testing observa el estado de la aplicación, razona sobre la información disponible, ejecuta una acción y posteriormente verifica el resultado antes de decidir cómo continuar.

Generación de casos de prueba

Para generar un caso de prueba se parte de un objetivo funcional.

El proceso planteado consiste en proporcionar una descripción funcional, observar la aplicación, permitir que el LLM seleccione una acción, hacer que Playwright ejecute esa acción y posteriormente permitir que el modelo observe el resultado obtenido.

De esta manera se establece una interacción continua entre objetivo funcional, modelo de lenguaje, Playwright y aplicación.

Adaptación ante cambios

La utilización de un modelo de lenguaje también puede aplicarse cuando una prueba deja de funcionar como consecuencia de una modificación de la interfaz.

El proceso planteado comienza cuando el test falla. A partir de ese momento se observa la interfaz actual junto con el objetivo original de la prueba. El LLM analiza el nuevo estado y propone una alternativa para localizar el elemento buscado. Posteriormente, Playwright vuelve a ejecutar la acción utilizando la información obtenida.

Esta estrategia se relaciona directamente con el concepto de QA adaptativo, ya que busca responder de manera controlada ante cambios de la aplicación.

Componentes principales

Dentro de la implementación también se trabajó con diferentes componentes vinculados a Playwright.

Los **Locators** se utilizan para identificar elementos de la interfaz. En los ejemplos de la actividad se emplean distintos `getByTestId()` para localizar campos, botones y elementos asociados con el estado de la aplicación.

El **Page Object Model** permite concentrar en clases las referencias a elementos y las operaciones correspondientes a una determinada pantalla. De esta forma, la lógica asociada a la interfaz queda separada de los casos de prueba y puede reutilizarse.

Las **fixtures** permiten preparar recursos y estados utilizados por los tests, mientras que **Trace Viewer** aporta información detallada para analizar posteriormente una ejecución. Ambos elementos aparecen incluidos dentro de los componentes utilizados en la actividad.

Finalmente, la presentación incorpora una complementación mediante **Playwright MCP**, incluyendo como usos comunes la navegación, interacción con elementos, inspección visual, control de entrada, gestión de diálogos y gestión de pestañas.

---

4. Implementación y ejecución de tests E2E

El cuarto módulo corresponde a la implementación concreta de las pruebas automatizadas.

La estrategia se apoya en el uso de selectores robustos mediante `page.getByTestId()`. Esto permite aislar los tests respecto de estilos CSS y reducir la utilización de selectores demasiado dependientes de la estructura visual.

También se utiliza `beforeEach` para realizar una navegación limpia hacia la raíz de la aplicación antes de cada prueba, buscando garantizar la independencia entre ejecuciones.

Las validaciones se implementan mediante `expect()`, comprobando aspectos como visibilidad y contenido dentro del DOM dinámico.

Casos de prueba y reactividad de la SPA

La suite desarrollada contempla diferentes comportamientos de la aplicación.

Se comprueba la validación de campos obligatorios cuando el formulario se envía vacío, junto con reglas específicas de formato para correo electrónico y contraseña.

También se prueba la interacción client-side mediante un botón que permite alternar la visibilidad de la contraseña.

Finalmente, se valida un flujo completo de registro: ingreso de datos, envío correcto, actualización reactiva del contador y limpieza de los campos sin necesidad de recargar la página.

Resultados de la suite

La suite fue ejecutada por consola mediante Playwright sobre Chromium.

Según los resultados presentados, se obtuvo una tasa de éxito del 100 %, con **8 tests aprobados en 16,9 segundos**, sin fallos de sincronización.

Además, Playwright generó automáticamente un HTML Report, utilizado para revisar el resultado general de la suite y analizar los tiempos asociados con las distintas pruebas y assertions.

Trace Viewer

Como herramienta complementaria para el diagnóstico se utilizó Trace Viewer.

Esta funcionalidad permite realizar una inspección visual de la ejecución paso a paso, observando el estado de la aplicación antes y después de cada acción.

También permite obtener trazabilidad mediante capturas secuenciales, llamadas de red e inspección del DOM durante las assertions, facilitando el análisis de errores y del comportamiento interno de una prueba.

---

5. Especificación de testing y selección de la SPA

Para la implementación se seleccionó una **SPA de registro de usuarios**.

La aplicación posee un formulario compuesto por nombre, apellido, correo electrónico y contraseña, junto con las acciones Registrar y Limpiar.

Sobre esta aplicación se busca comprobar el comportamiento general del formulario, las validaciones con datos válidos e inválidos, la confirmación luego de un registro correcto, la actualización de la lista de usuarios y la actualización del contador dentro de la misma SPA.

Contexto de la aplicación

La SPA funciona como un campo de pruebas para automatización.

El usuario completa los datos del formulario y la aplicación valida la información ingresada. Cuando el registro es válido, se genera una confirmación.

El resultado esperado consiste en que el usuario aparezca dentro de la lista, que el contador refleje correctamente la cantidad de usuarios registrados y que los campos del formulario queden vacíos después de completar el registro. Además, el botón Limpiar debe permitir restaurar el formulario.

Flujos críticos y criterios de aceptación

Los flujos incluidos en las pruebas comprenden la carga inicial de la SPA, el envío del formulario vacío, la validación del nombre y apellido, la validación del correo electrónico, la validación de la contraseña, la función de mostrar u ocultar la contraseña, el registro exitoso y el funcionamiento del botón Limpiar.

Para el flujo de registro se establecieron como criterios de aceptación que se muestre la confirmación correspondiente, que los datos registrados coincidan con los ingresados, que el contador se actualice, que el usuario aparezca en la lista y que los campos del formulario queden vacíos al finalizar.

Restricciones técnicas y datos de prueba

La ejecución se realizó utilizando Chromium con la configuración Desktop Chrome de Playwright. El viewport corresponde a dicha configuración sin establecer un tamaño personalizado y la aplicación se ejecuta localmente.

Como datos válidos se utilizaron el nombre Pablo, apellido Gómez, el correo `pablo.gomez@empresa.com` y la contraseña `ClaveSegura2026`.

También se definieron datos inválidos para comprobar las reglas de validación, como nombres o apellidos demasiado cortos o con caracteres no permitidos, direcciones de correo con formato incorrecto y contraseñas que no cumplen los requisitos esperados.

Conclusión

El desarrollo de la actividad permitió analizar Playwright no solamente como una herramienta capaz de automatizar acciones dentro de un navegador, sino como parte de una estrategia de testing orientada a aplicaciones web modernas.

Las Single Page Applications presentan desafíos particulares debido a la modificación dinámica del DOM, la existencia de contenido asincrónico y la navegación mediante rutas del lado del cliente. Frente a estas situaciones, el uso de Locators robustos, mecanismos de sincronización y assertions permite desarrollar pruebas más estables y mantenibles.

El concepto de QA adaptativo amplía esta estrategia al considerar la posibilidad de responder de manera controlada ante cambios en la interfaz. En este contexto, los modelos de lenguaje pueden complementar a Playwright mediante el análisis del estado de la aplicación, la propuesta de acciones y la adaptación de determinadas pruebas.

La implementación práctica permitió aplicar estos conceptos sobre una SPA de registro de usuarios, definiendo flujos funcionales, criterios de aceptación, restricciones técnicas y datos de prueba. La ejecución final alcanzó ocho pruebas aprobadas, acompañadas por evidencias generadas mediante HTML Report y Trace Viewer.

En conjunto, la actividad muestra cómo Playwright puede utilizarse para desarrollar una estrategia de testing End-to-End que combine automatización, robustez de selectores, análisis de aplicaciones dinámicas, herramientas de diagnóstico y mecanismos de adaptación orientados a mejorar la calidad y mantenibilidad de las pruebas.